

Module: `llm_orchestrator/`

LLM Orchestration Layer

Position in Architecture



This module is the intelligence layer. It reads from the concept library and the current runtime state, calls the external LLM API, and produces two outputs simultaneously: a structured animation plan (`AnimationIntent`) and a structured explanation text (`NarrativeBlock`). It owns the retry/feedback loop when the Translator rejects an `AnimationIntent`.

Directory Layout

llm_orchestrator/	
— mod.rs	# Public API, LLMOrchestrator struct
— request.rs	# OrchestratorRequest — input to the orchestrator
— response.rs	# OrchestratorResponse — structured LLM output
— animation_intent.rs	# AnimationIntent — structured animation plan (Translator input)
— narrative.rs	# NarrativeBlock — structured explanation text (Bus input)
— prompt_builder.rs	# LLMPromptBuilder — constructs LLM API prompts
— response_parser.rs	# LLMResponseParser — parses raw LLM text into structs
— context_builder.rs	# ConceptContextBuilder — assembles concept context
— feedback_loop.rs	# FeedbackLoop — retry logic on Translator failure
— llm_client.rs	# LLMClient trait + AnthropicClient implementation

File: `llm_orchestrator/request.rs`

Purpose

`OrchestratorRequest` is the complete input package the orchestrator receives from the session when a new step begins or a concept section changes.

What it contains

`OrchestratorRequest` struct:

- `session_id: String` — identifies the user session, used for cache keying and log correlation.
- `concept_id: String` — the concept currently being taught.
- `section_id: String` — the specific `ConceptSection` within the concept page that is now active.
- `step_index: usize` — which derivation step within the current section is being entered. `0` means the section is just starting.
- `equation_id: Option<String>` — the `NamedEquation` being explained at this step, if one is active.
- `active_node_id: Option<String>` — the `node_id` of the specific `MathExpression` sub-expression that the step trace just evaluated. `None` when entering a new section rather than advancing a step within an equation.
- `current_variable_bindings: Vec<(String, f64)>` — all current variable values from `RuntimeState.world.parameters`, passed so the LLM can include concrete numbers in its explanation.
- `last_step_description: Option<String>` — the `description` from the most recently emitted `MathStepEvent`, giving the LLM context on what the engine just computed.
- `user_action: UserAction` — what triggered this request.
- `is_first_view: bool` — true if this is the first time this user has seen this step (cache miss), false if they are re-visiting (cache hit — response is served from cache without an LLM call).

`UserAction` enum:

- `AdvanceStep` — user pressed Next Step
 - `RewindStep` — user pressed Previous Step
 - `EnterSection { section_id: String }` — user navigated to a new section
 - `VariableChanged { name: String, new_value: f64 }` — user moved a slider
 - `AskQuestion { text: String }` — user typed a question (extension point)
-

File: `llm_orchestrator/animation_intent.rs`

Purpose

`AnimationIntent` is the structured animation plan the LLM produces. It uses the controlled vocabulary from `EntityRoleRegistry` and `MotionVerbMap` (both in `llm_dsl_translator/`) so every intent the LLM outputs is translatable to DSL. This is the primary input to the `llm_dsl_translator` module.

What it contains

`AnimationIntent` struct:

- `entities: Vec<EntityIntent>` — the geometric objects that should exist in the scene.
- `motions: Vec<MotionIntent>` — what moves, how, and when.
- `annotations: Vec<AnnotationIntent>` — text labels to display in Panel C.
- `highlights: Vec<HighlightIntent>` — which entity highlights at which step and with which color.
- `camera: CameraIntent` — where the camera should be positioned and focused.
- `concept_id: String` — for `concept_ref` in the generated DSL.
- `section_id: String`
- `step_index: usize`

`EntityIntent` struct:

- `role: String` — a role name from the `EntityRoleRegistry`, e.g. `"unit_circle"`, `"radius_vector"`, `"x_axis_projection"`, `"angle_arc"`. The Translator looks up the entity template for this role.
- `entity_id: String` — the name this entity will have in the generated DSL, e.g. `"radius_vec"`. Must be valid DSL identifier characters.
- `initial_position: Option<[f64; 3]>` — overrides the template default if provided.
- `initial_scale: Option<[f64; 3]>` — overrides the template default.
- `color_override: Option<String>` — hex color, overrides the template material.
- `represents_variable: Option<String>` — the parameter name this entity's geometry encodes (e.g., `"theta"` for an angle arc whose size represents the angle). Written into the DSL `variable_display` block.

`MotionIntent` struct:

- `verb: String` — a verb from the `MotionVerbMap`, e.g. `"rotate"`, `"sweep_to"`, `"scale_to"`, `"fade_in"`, `"oscillate"`. The Translator maps this to a DSL `motion` type.
- `target_entity_id: String` — which entity moves.

- `parameters: HashMap<String, serde_json::Value>` — motion-specific parameters. Keys and value types depend on the verb (specified in the `MotionVerbMap`). Examples: `{"axis": [0,0,1], "from_angle": 0, "to_angle": 6.28}` for rotate; `{"duration_seconds": 3.0}`.
- `start_time: f64` — timeline start in seconds.
- `duration: f64` — timeline duration in seconds.
- `compound_with: Vec<String>` — if non-empty, this motion is part of a `compound_motion` block with the listed motion IDs.

AnnotationIntent struct:

- `label_latex: String` — the label text, may contain LaTeX.
- `anchor_entity_id: String` — which entity to attach to.
- `offset: [f64; 3]` — world-space offset from entity center.
- `show_at_step: usize` — the step index at which this annotation becomes visible. Mapped to a timeline timestamp by the Translator.
- `equation_node_id: Option<String>` — the expression node this annotation references, for synchronized highlighting.

HighlightIntent struct:

- `entity_id: String`
- `at_step: usize` — conceptual step index (not a timeline timestamp; Translator converts).
- `highlight_token: String` — the token this highlight corresponds to.
- `color_index: u8` — 0–15.

CameraIntent struct:

- `position: Option<[f64; 3]>` — if `None`, auto-frame.
- `target: Option<[f64; 3]>` — look-at point.
- `auto_frame_entity_id: Option<String>` — if set, camera auto-frames to show this entity prominently at each new highlight.

File: `llm_orchestrator/narrative.rs`

Purpose

`NarrativeBlock` is the structured explanation text produced by the LLM for the current step. It is the primary

input to the `explanation_bus` for Panel A.

What it contains

NarrativeBlock struct:

- `section_id: String`
- `step_index: usize`
- `segments: Vec<NarrativeSegment>` — the prose broken into segments where each segment may reference an equation term.

NarrativeSegment struct:

- `text: String` — the prose text for this segment. LaTeX is allowed between `$...$` delimiters.
- `referenced_node_id: Option<String>` — if this segment of text is explaining a specific sub-expression, this is its `node_id`. The Narrative Panel highlights the corresponding equation term in Panel B when this segment is being read.
- `referenced_highlight_token: Option<String>` — if the node has a highlight token, this is set so the panel can color the text segment with the token's color.
- `is_rule_explanation: bool` — true if this segment explains a mathematical rule (e.g., "The chain rule states..."). These segments get a distinct visual treatment in the panel.

StepAnnotationMap struct:

- `annotations: HashMap<String, StepAnnotationDetail>` — maps `highlight_token` to a detailed explanation of that term for this step.

StepAnnotationDetail struct (shown in the "Why this step?" drawer):

- `term_name: String` — "outer function", "inner derivative", etc.
 - `explanation: String` — 2–4 sentences explaining what this term represents and why it matters in the current step.
 - `rule_name: Option<String>` — the mathematical rule being applied.
 - `rule_definition_latex: Option<String>` — the rule formula itself, shown in the drawer.
-

File: `llm_orchestrator/response.rs`

Purpose

`OrchestratorResponse` is the complete parsed output of one LLM call.

What it contains

`OrchestratorResponse` struct:

- `session_id: String`
 - `concept_id: String`
 - `section_id: String`
 - `step_index: usize`
 - `animation_intent: AnimationIntent` — passed to the `llm_dsl_translator`.
 - `narrative: NarrativeBlock` — passed to the `explanation_bus`.
 - `step_annotations: StepAnnotationMap` — per-token "Why this step?" content.
 - `highlight_token_assignments: Vec<(String, u8)>` — `(token_string, color_index)` pairs to register in `HighlightTokenRegistry` (in `step_trace/`).
 - `concept_jump: Option<String>` — if the LLM determines the user needs to review a prerequisite first, this is the prerequisite concept ID. The session uses this to navigate backward.
 - `llm_model: String` — which model was used, stored for logging.
 - `latency_ms: u64` — API call duration, for monitoring.
-

File: `llm_orchestrator/prompt_builder.rs`

Purpose

`LLMPromptBuilder` constructs the system prompt and user message for each LLM API call. It encodes the LLM's complete "job description" and all context needed to produce a valid response.

What it contains

`LLMPromptBuilder` struct:

- `entity_role_registry_json: String` — pre-serialized JSON of the `EntityRoleRegistry` from `llm_dsl_translator/`. Embedded in every system prompt so the LLM knows what objects it can use.
- `motion_verb_map_json: String` — pre-serialized JSON of the `MotionVerbMap`. Same reason.

- `color_table_json: String` — pre-serialized `ColorTable` from `step_trace/`. So the LLM assigns valid color indices.
- `response_schema_json: String` — JSON Schema for `AnimationIntent`. The LLM is instructed to produce output conforming to this schema.
- `few_shot_examples: Vec<FewShotExample>` — 3–5 pre-written examples of (concept context → LLM response). These are the most important prompt engineering artifacts: they show the LLM exactly what format is expected.

`FewShotExample` struct:

- `concept_summary: String` — a brief description of the concept (not the full context, just enough to establish the scenario)
- `request_json: String` — a serialized `OrchestratorRequest` (or summary)
- `ideal_response: String` — the ideal LLM output for this request, in the exact two-part format the system expects

`LLMPromptBuilder::build_system_prompt(&self) -> String` — returns the invariant system prompt. Contains:

1. Role definition: "You are an expert mathematics educator who designs animated visualizations and writes clear explanations simultaneously. You always produce both the animation plan and the explanation for the same mathematical step at the same time."
2. The response format contract: "You must respond in exactly two parts. Part 1: A JSON object conforming to the AnimationIntent schema, delimited by `---ANIMATION_INTENT_START---` and `---ANIMATION_INTENT_END---`. Part 2: Explanation text in the NarrativeBlock format, delimited by `---NARRATIVE_START---` and `---NARRATIVE_END---`."
3. The entity role registry embedded as a JSON block.
4. The motion verb map embedded as a JSON block.
5. The color table embedded as a JSON block.
6. The AnimationIntent JSON schema.
7. The NarrativeBlock format specification with example markup.
8. The few-shot examples.

`LLMPromptBuilder::build_user_message(&self, request: &OrchestratorRequest, context: &ConceptContext) -> String` — returns the per-request user message. Contains:

1. The current `ConceptPage` section prose (from `ConceptContext.section_body`).
2. The `NamedEquation` with all `TermAnnotation` objects listed.
3. The current step index and which sub-expression is active (from `request.active_node_id`).

4. The current variable bindings as a table.
 5. The last step description (from `request.last_step_description`).
 6. Any special instructions for this request type (e.g., for `UserAction::VariableChanged`, "The user changed {variable} to {value}. Explain how this affects the visualization and update the animation to reflect the new value.").
-

File: `llm_orchestrator/response_parser.rs`

Purpose

`LLMResponseParser` extracts and validates the two structured parts from the LLM's raw text output.

What it contains

`LLMResponseParser` struct — stateless, all methods are `fn`.

`parse(raw_text: &str, request: &OrchestratorRequest) -> Result<OrchestratorResponse, ParseError>`:

1. Splits the text on the delimiter strings to extract the JSON block and the narrative text block.
2. Strips any stray markdown fences (````json`) from the JSON block.
3. Calls `serde_json::from_str::<AnimationIntentRaw>` to parse the JSON.
4. Validates the parsed `AnimationIntentRaw` against the schema (checking required fields, enum validity for role names and verb names against the registries).
5. Calls `parse_narrative(narrative_text)` to extract `NarrativeSegment` objects.
6. Calls `extract_highlight_assignments(animation_intent)` to collect token/color pairs.
7. Assembles the `OrchestratorResponse`.

`parse_narrative(text: &str) -> Vec<NarrativeSegment>` — scans the narrative text for the term reference markup `{{term:node_id:token}}` and splits the text into `NarrativeSegment` objects, with and without `referenced_node_id` set.

`ParseError` enum:

- `MissingDelimiter { expected: String }` — the LLM didn't use the required delimiters
- `InvalidJson { reason: String }` — the JSON block is malformed
- `UnknownEntityRole { role: String }` — references a role not in the registry
- `UnknownMotionVerb { verb: String }` — references a verb not in the verb map
- `InvalidColorIndex { index: u8 }` — color index > 15

- `MissingNarrativeSection` — narrative delimiter missing
-

File: `llm_orchestrator/context_builder.rs`

Purpose

`ConceptContextBuilder` assembles the full context package the prompt builder needs, from the concept library and runtime state.

What it contains

`ConceptContext` struct:

- `concept: Concept`
- `current_page: ConceptPage`
- `current_section: ConceptSection`
- `active_equation: Option<NamedEquation>`
- `active_term_annotation: Option<TermAnnotation>` — the annotation for `request.active_node_id`, if one exists.
- `prerequisite_summaries: Vec<String>` — one-sentence summaries of the most immediate prerequisite concepts (to give the LLM background without blowing the context window).
- `derivation_chain_summary: Vec<String>` — one-sentence descriptions of each step in the current equation's derivation chain up to the current step.

`ConceptContextBuilder::build(request: &OrchestratorRequest, library: &LibraryStore) ->`

`Result<ConceptContext, ContextError>` — reads from the library to assemble the context. Includes a `fit_to_context_window()` step that truncates prerequisite summaries and derivation chain if the total token estimate would exceed a configurable limit (default: 4000 tokens of context, leaving room for the prompt infrastructure and the LLM response).

File: `llm_orchestrator/feedback_loop.rs`

Purpose

`FeedbackLoop` handles the case where the `TranslationValidator` in `llm_dsl_translator/` rejects the LLM's `AnimationIntent`. It formats the errors as a correction prompt and re-calls the LLM.

What it contains

`FeedbackLoop` struct:

- `max_retries: usize` — default 3.
- `client: Arc<dyn LLMClient>` — the same client used by the orchestrator.
- `prompt_builder: Arc<LLMPromptBuilder>`

`FeedbackLoop::correct(original_intent: &AnimationIntent, errors: &[TranslationError], request: &OrchestratorRequest, context: &ConceptContext) -> Result<AnimationIntent, FeedbackError>`:

1. Formats `errors` as a human-readable list (e.g., "Error: entity role 'unit_cricle' is not in the EntityRoleRegistry. Did you mean 'unit_circle'?").
2. Constructs a correction prompt that includes the original intent JSON, the error list, and an instruction to fix only the identified problems.
3. Calls the LLM client.
4. Parses only the `AnimationIntent` part of the response (not the narrative, which is unchanged).
5. Returns the corrected intent or `FeedbackError::MaxRetriesExceeded` if after `max_retries` attempts the intent still fails validation.

`FeedbackError` enum:

- `MaxRetriesExceeded { last_errors: Vec<TranslationError> }` — all retries exhausted; the session falls back to a pre-authored static scene.
- `LLMError(LLMClientError)` — the API itself failed.
- `ParseError(ParseError)` — the correction response was malformed.

File: `llm_orchestrator/llm_client.rs`

Purpose

`LLMClient` is the pluggable interface for LLM API calls. Separates the API call mechanics from the orchestration logic.

What it contains

`LLMClient` trait:

```
trait LLMClient: Send + Sync {
    async fn complete(&self, system: &str, user: &str, max_tokens: usize) -> Result<String, LLMClientError>;
    fn model_name(&self) -> &str;
}
```

AnthropicClient struct (primary implementation):

- `api_key: String`
- `model: String` — e.g. `"claude-sonnet-4-6"`
- `http_client: reqwest::Client`

AnthropicClient::complete implementation:

1. Builds the JSON body for the Anthropic `/v1/messages` endpoint.
2. Sends via `request`.
3. Reads `response.content[0].text`.
4. Returns the text or a typed error.

LLMClientError enum:

- `ApiError { status: u16, body: String }`
- `NetworkError(String)`
- `Timeout`
- `RateLimited { retry_after_seconds: u64 }`

CachingLLMClient struct (wrapper):

- `inner: Arc<dyn LLMClient>`
- `cache: Arc<RwLock<HashMap<String, String>>>` — maps `(system_hash, user_hash)` to response text.

The cache key is a SHA-256 hash of the concatenated system and user prompt. Since prompts include the `step_index` and `section_id`, identical concept views produce cache hits. Cache is per-server-process (not persisted across restarts in the MVP; Redis integration is a later enhancement).

File: `llm_orchestrator/mod.rs`

Purpose

`LLMOrchestrator` — the public-facing struct — and orchestration logic.

What it contains

LLMOrchestrator struct:

- `library: Arc<LibraryStore>`

- `client: Arc<dyn LLMClient>`
- `prompt_builder: Arc<LLMPromptBuilder>`
- `feedback_loop: FeedbackLoop`
- `translator_validator: Arc<TranslationValidator>` — injected reference to the Translator's validation component, used by `feedback_loop`.

`LLMOrchestrator::process(request: OrchestratorRequest) -> Result<OrchestratorResponse, OrchestratorError>`:

1. Build `ConceptContext` via `ConceptContextBuilder::build`.
2. Check cache: if `request.is_first_view == false`, return cached response.
3. Build system prompt and user message via `LLMPromptBuilder`.
4. Call `client.complete()`.
5. Parse response via `LLMResponseParser::parse`.
6. Validate `AnimationIntent` via `TranslationValidator::validate_intent`. If validation fails, enter `FeedbackLoop::correct` retry cycle.
7. Cache the validated response.
8. Return `OrchestratorResponse`.

`OrchestratorError` enum:

- `LibraryError(LibraryError)`
- `LLMError(LLMClientError)`
- `ParseError(ParseError)`
- `FeedbackFailed(FeedbackError)` — all retries exhausted; caller must use fallback scene.
- `ContextBuildError(ContextError)`

Dependencies Summary

Depends on	Why
<code>concept_library/</code>	Reads all concept, equation, section, page data for prompt building
<code>step_trace/</code>	Reads <code>ColorTable</code> for color index validation; receives <code>active_node_id</code> and <code>last_step_description</code> from active trace
<code>llm_dsl_translator/</code>	<code>TranslationValidator.validate_intent()</code> — used in the feedback loop

Depends on	Why
<code>request</code> (async HTTP)	LLM API calls
<code>serde_json</code>	Request/response JSON handling
<code>sha2</code>	Cache key hashing

Consumed by

Consumer	What it reads
<code>server/session.rs</code>	Calls <code>LLMOrchestrator::process()</code> on each step change
<code>llm_dsl_translator/</code>	Receives <code>AnimationIntent</code> from <code>OrchestratorResponse</code>
<code>explanation_bus/</code>	Receives <code>NarrativeBlock</code> and <code>StepAnnotationMap</code> from <code>OrchestratorResponse</code>
<code>step_trace/</code>	Receives <code>highlight_token_assignments</code> to register in <code>HighlightTokenRegistry</code>